Архитектура

Буферный кеш и журнал



Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Устройство буферного кеша

Алгоритм вытеснения

Журнал предзаписи

Контрольная точка

Процессы, связанные с буферным кешем и журналом

Буферный кеш

Массив буферов

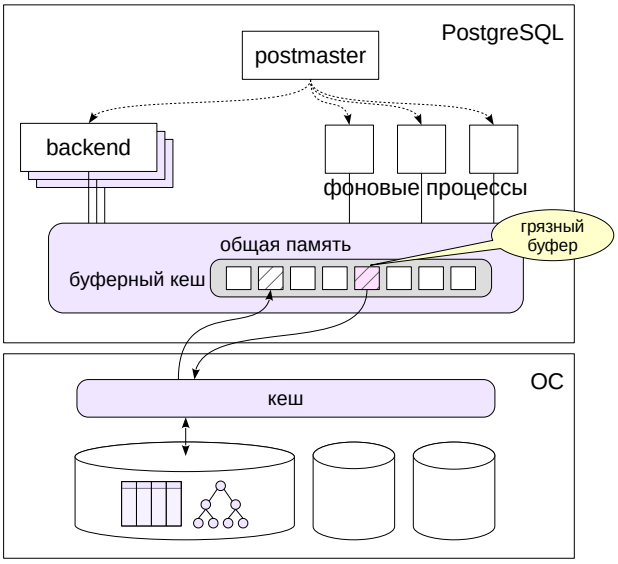
- страница данных (8 Кбайт)
- доп. информация

«Грязные» буферы

- асинхронная запись

Блокировки в памяти

- для совместного доступа



Буферный кеш используется для сглаживания скорости работы оперативной памяти и дисков. Он состоит из массива буферов, которые содержат страницы данных и дополнительную информацию (например, имя файла и положение страницы внутри этого файла).

Размер страницы обычно составляет 8 Кбайт; размер можно изменить только при сборке PostgreSQL.

Любая работа со страницами данных проходит через буферный кеш. Если какой-либо процесс собирается работать со страницей, он в первую очередь пытается найти ее в кеше. Если страницы нет, он обращается к операционной системе с просьбой прочитать эту страницу и помещает ее в буферный кеш. (Обратите внимание, что ОС может прочитать страницу с диска, а может обнаружить ее в собственном кеше.)

После того, как страница записана в буферный кеш, к ней можно обращаться многократно без накладных расходов на вызовы ОС.

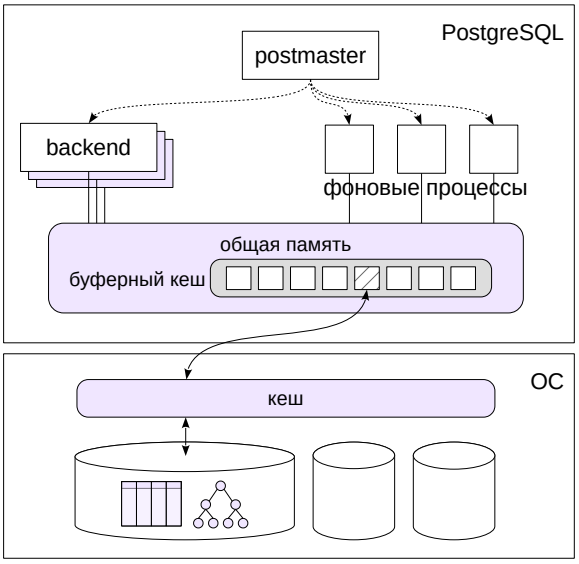
Если процесс изменил данные в странице, соответствующий буфер становится «грязным». Измененная страница подлежит записи на диск, но по соображениям производительности запись происходит асинхронно и может откладываться.

Буферный кеш, как и другие структуры общей памяти, защищен блокировками для управления одновременным доступом. Хотя блокировки и реализованы эффективно, доступ к буферному кешу далеко не так быстр, как простое обращение к оперативной памяти. Поэтому в общем случае чем меньше данных читает и изменяет запрос, тем быстрее он будет работать.

Вытеснение

Вытеснение редко используемых страниц

- грязный буфер записывается на диск
- на освободившееся место читается другая страница



Размер буферного кеша обычно не так велик, чтобы база данных помещалась в него целиком. Его ограничивают и доступная оперативная память, и возрастающие при его увеличении накладные расходы. Поэтому при чтении очередной страницы рано или поздно окажется, что место в буферном кеше закончилось. В этом случае применяется *вытеснение* страниц.

Алгоритм вытеснения выбирает в кеше страницу, которая в последнее время использовалась реже других. Если выбранный буфер оказался грязным, страница записывается на диск, чтобы не потерять сделанные в ней изменения. Затем в освободившийся буфер читается новая страница.

Такой алгоритм вытеснения называется LRU — Least Recently Used. Он сохраняет в кеше данные, с которыми происходит активная работа. Таких «горячих» данных обычно не слишком много, и при достаточном объеме буферного кеша получается существенно сократить количество обращений к ОС (и дисковых операций).

Влияние буферного кеша на выполнение запросов

Создаем новую БД в кластере и подключаемся к ней (подробнее про базы данных — в модуле «Организация данных»):

```
=> CREATE DATABASE arch_wal_overview;
```

```
CREATE DATABASE
```

```
=> \c arch_wal_overview
```

```
You are now connected to database "arch_wal_overview" as user "student".
```

Создадим таблицу:

```
=> CREATE TABLE t(n integer);
```

```
CREATE TABLE
```

Заполним таблицу некоторым количеством строк:

```
=> INSERT INTO t SELECT id FROM generate_series(1,100_000) AS id;
```

```
INSERT 0 100000
```

Размер буферного кеша показывает параметр shared_buffers:

```
=> SHOW shared_buffers;
```

```
 shared_buffers
----------------
 128MB
(1 row)
```

Значение по умолчанию слишком мало; в любой реальной системе его требуется увеличить сразу после установки сервера (изменение требует перезапуска).

Теперь перезапустим сервер, чтобы содержимое буферного кеша сбросилось.

```
=> \q
```

```
student$ sudo pg_ctlcluster 16 main restart
```

```
student$ psql arch_wal_overview
```

Сравним, что происходит при первом и при втором выполнении одного и того же запроса. В этом курсе мы не рассматриваем подробно планы запросов, но иногда будем в них заглядывать. Сейчас мы воспользуемся командой EXPLAIN ANALYZE, которая выполняет запрос и выводит не только план выполнения, но и дополнительную информацию:

```
=> EXPLAIN (analyze, buffers, costs off, timing off)
SELECT * FROM t;
```

```
                QUERY PLAN
-------------------------------------------
 Seq Scan on t (actual rows=100000 loops=1)
   Buffers: shared read=443
 Planning:
   Buffers: shared hit=12 read=8 dirtied=1
 Planning Time: 0.146 ms
 Execution Time: 6.177 ms
(6 rows)
```

Строка «Buffers: shared» показывает использование буферного кеша.

- read — количество буферов, в которые пришлось прочитать страницы с диска.

```
=> EXPLAIN (analyze, buffers, costs off, timing off)
SELECT * FROM t;
```

```
                QUERY PLAN
-------------------------------------------
 Seq Scan on t (actual rows=100000 loops=1)
   Buffers: shared hit=443
 Planning Time: 0.028 ms
 Execution Time: 4.306 ms
(4 rows)
```

- hit — количество буферов, в которых нашлись нужные для запроса страницы.

Обратите внимание, что во второй раз уменьшилось и время выполнения запроса, и время его планирования (потому что таблицы системного каталога тоже кешируются).

Журнал предзаписи (WAL)

Проблема: при сбое теряются данные из оперативной памяти, не записанные на диск

Журнал

- поток информации о выполняемых действиях, позволяющий повторно выполнить потерянные при сбое операции
- запись попадает на диск раньше, чем измененные данные

Журнал защищает

- страницы таблиц, индексов и других объектов
- статус транзакций (clog)

Журнал не защищает

- временные и нежурналируемые таблицы

Наличие буферного кеша (и других буферов в оперативной памяти) увеличивает производительность, но уменьшает надежность.
В случае сбоя в СУБД содержимое буферного кеша потеряется. Если сбой произойдет в операционной системе или на аппаратном уровне, то пропадет содержимое и буферов ОС (но с этим справляется сама операционная система).

Для обеспечения надежности PostgreSQL использует журналирование. При выполнении любой операции формируется запись, содержащая минимально необходимую информацию для того, чтобы операцию можно было выполнить повторно. Такая запись должна попасть на диск (или другой энергонезависимый накопитель) раньше, чем будут записаны изменяемые операцией данные (поэтому журнал и называется *журналом предзаписи*, write-ahead log).

Файлы журнала располагаются в каталоге PGDATA/pg_wal.

Журнал защищает все объекты, работа с которыми ведется в оперативной памяти: таблицы, индексы и другие объекты, статус транзакций.

В журнал не попадают данные о *временных таблицах* (такие таблицы доступны только создавшему их пользователю и только на время сеанса или транзакции) и о *нежурналируемых таблицах* (такие таблицы ничем не отличаются от обычных, кроме того, что не защищены журналом). В случае сбоя нежурналируемые таблицы просто очищаются, зато работа с ними происходит быстрее.

https://postgrespro.ru/docs/postgresql/16/wal-intro

Журнал предзаписи

Логически журнал можно представить в виде непрерывного потока записей. Каждая запись имеет номер, называемый LSN (Log Sequence Number). Это 64-разрядное число — смещение записи в байтах относительно начала журнала.

Текущую позицию показывает функция pg_current_wal_lsn:

```
=> SELECT pg_current_wal_lsn();
```

```
 pg_current_wal_lsn
--------------------
 0/23BCD28
(1 row)
```

Позиция записывается как два 32-разрядных числа через косую черту. Запомним это значение.

Выполним теперь какие-нибудь операции и посмотрим, как изменилась позиция.

```
=> UPDATE t SET n = 100_001 WHERE n = 1;
```

```
UPDATE 1
```

```
=> SELECT pg_current_wal_lsn();
```

```
 pg_current_wal_lsn
--------------------
 0/23BFD40
(1 row)
```

Интересны не абсолютные числа, а их разница, которая показывает размер сгенерированных журнальных записей в байтах:

```
=> SELECT '0/23BFD40'::pg_lsn - '0/23BCD28'::pg_lsn AS bytes;
```

```
 bytes
-------
 12312
(1 row)
```

Физически журнал хранится в файлах в отдельном каталоге (PGDATA/pg_wal). По умолчанию файлы имеют размер 16 Мбайт, задать другой размер можно при инициализации кластера баз данных.

На файлы можно взглянуть не только средствами файловой системы, но и с помощью функции:

```
=> SELECT * FROM pg_ls_waldir() ORDER BY name LIMIT 10;
```

```
           name           |   size   |      modification
--------------------------+----------+------------------------
 000000010000000000000002 | 16777216 | 2025-06-24 04:30:27+03
 000000010000000000000003 | 16777216 | 2025-06-24 04:30:18+03
(2 rows)
```

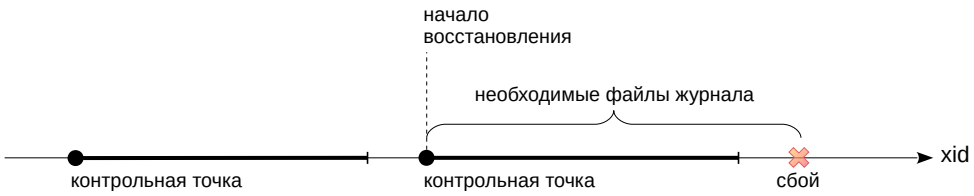
Контрольная точка

Периодический сброс всех грязных буферов на диск

- гарантирует попадание на диск всех изменений до контрольной точки
- ограничивает размер журнала, необходимого для восстановления

Восстановление при сбое

- начинается с последней контрольной точки
- последовательно проигрываются записи, если изменений нет на диске



Когда сервер PostgreSQL запускается после сбоя, он входит в режим восстановления. Информация на диске в это время несогласованна: изменения «горячих» страниц пропали, поскольку эти страницы все еще находились в кеше, а более поздние изменения уже были сброшены на диск.

Чтобы восстановить согласованность, PostgreSQL читает WAL и последовательно проигрывает каждую его запись, если соответствующее изменение не попало на диск. Таким образом восстанавливается работа всех транзакций. Транзакции, запись о фиксации которых не успела попасть в журнал, считаются оборванными.

Однако объем журнала за время работы сервера мог бы достигнуть гигантских размеров. Хранить его целиком и просматривать при сбое совершенно не реально. Поэтому СУБД периодически выполняет контрольную точку (КТ): принудительно сбрасывает на диск все грязные буферы (включая буферы clog, хранящие состояние транзакций).

Собственно «точка» — это момент начала записи буферов, которые были грязными на этот момент. Но точка считается выполненной только после того, как все такие буферы записаны. Это гарантирует, что все изменения, сделанные до момента КТ, находятся на диске.

В реальных системах с большим буферным кешем КТ может сбрасывать много грязных буферов, поэтому сервер распределяет процесс записи по времени, чтобы сгладить нагрузку на ввод-вывод.

Восстановление после сбоя начинается с момента последней завершенной КТ, что позволяет хранить только файлы журнала, записанные после нее.

Восстановление при помощи журнала

Измененные табличные страницы находятся в буферном кеше, но еще не записаны на диск. При обычной остановке сервер выполняет контрольную точку, чтобы записать все грязные страницы на диск, но мы сымитируем сбой системы, послав сигнал процессу postmaster.

```
student$ sudo head -n 1 /var/lib/postgresql/16/main/postmaster.pid
```

```
7700
```

```
student$ sudo kill -QUIT 7700
```

При старте произойдет восстановление согласованности данных с помощью журнала. Проверим:

```
student$ sudo pg_ctlcluster 16 main start
```

```
student$ psql arch_wal_overview
```

```
=> SELECT min(n), max(n) FROM t;
```

```
 min |  max
-----+--------
   2 | 100001
(1 row)
```

Все изменения были восстановлены.

PostgreSQL автоматически удаляет журнальные файлы, не требующиеся для восстановления, после выполнения контрольной точки.

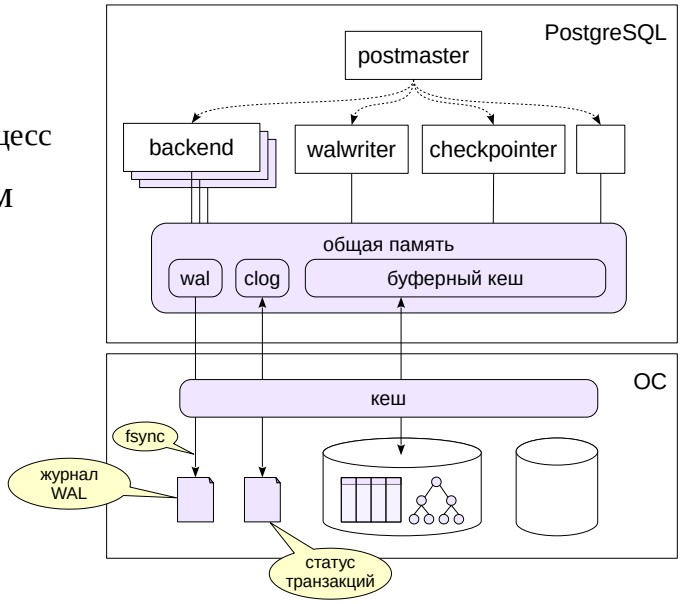
Производительность

Синхронный режим

- запись при фиксации
- обслуживающий процесс

Асинхронный режим

- фоновая запись
- walwriter



Механизм журналирования более эффективен, чем работа напрямую с диском без буферного кеша. Во-первых, размер журнальных записей меньше, чем размер целой страницы данных; во-вторых, журнал записывается строго последовательно (и обычно не читается, пока не случится сбой), с чем вполне справляются простые HDD-диски.

На эффективность можно также влиять настройкой. Если запись происходит сразу (синхронно), то гарантируется, что зафиксированная транзакция не пропадет. Но запись — довольно дорогая операция, в течение которой обслуживающий процесс, выполняющий фиксацию, вынужден ждать. Чтобы журнальная запись не «застряла» в кеше операционной системы, выполняется системный вызов fsync: PostgreSQL полагается на то, что это гарантирует попадание данных на энергонезависимый носитель.

Поэтому есть и режим отложенной (асинхронной) записи. В этом случае записи пишутся фоновым процессом walwriter постепенно, с небольшой задержкой. Надежность уменьшается, зато производительность увеличивается. Но и в этом случае после сбоя гарантируется восстановление согласованности.

На самом деле оба режима работают совместно. Журнальные записи долгой транзакции будут записываться асинхронно (чтобы освободить буферы WAL). А если при сбросе страницы данных окажется, что соответствующая журнальная запись еще не на диске, она тут же будет записана в синхронном режиме.

Основные процессы

Запись журнала

Контрольная точка

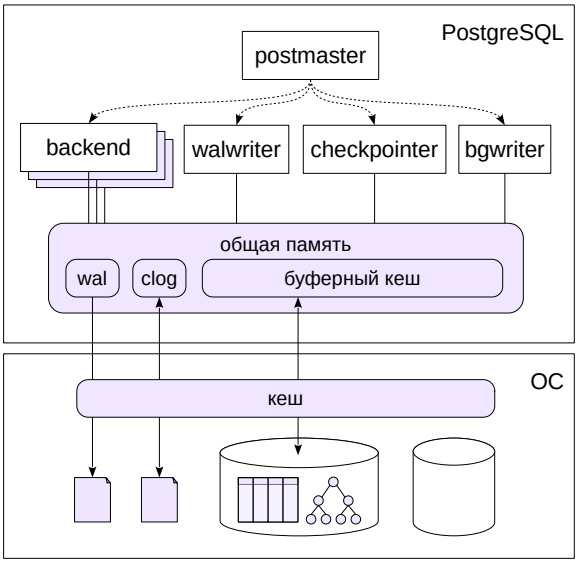
- сброс всех грязных буферов

Фоновая запись

- сброс части грязных буферов

Обслуживающие процессы

- сброс вытесняемого грязного буфера



Вернемся еще раз к процессам, связанным с обслуживанием буферного кеша и журнала.

Во-первых, это процесс walwriter, занимающийся асинхронной записью журнала на диск. При синхронном режиме записью журнала занимается тот процесс, который выполняет фиксацию транзакции.

Во-вторых, процесс контрольной точки checkpointer, периодически сбрасывающий все грязные буферы на диск.

В-третьих, процесс фоновой записи background writer (или bgwriter). Этот процесс похож на процесс контрольной точки, но записывает только часть грязных буферов, причем те, которые с большой вероятностью будут вытеснены в ближайшее время. Таким образом, когда обслуживающий процесс выберет буфер, чтобы прочитать новую страницу, старая страница скорее всего уже не будет грязной и не надо будет тратить время, чтобы сбросить ее на диск.

И в-четвертых, обслуживающие процессы, читающие данные в буферный кеш. Если, несмотря на работу процессов контрольной точки и фоновой записи, вытесняемый буфер окажется грязным, обслуживающий процесс самостоятельно запишет его на диск.

Уровни журнала

Minimal

гарантия восстановления после сбоя

Replica *(по умолчанию)*

резервное копирование

репликация: передача и проигрывание журнала на другом сервере

Logical

логическая репликация: информация о добавлении, изменении и удалении табличных строк

Как уже говорилось, причиной появления журнала является необходимость защищать информацию от сбоев из-за потери содержимого оперативной памяти.

Однако журнал — механизм, который оказалось удобно применять и для других целей, если добавить в него дополнительную информацию.

Объем данных, который попадает в журнал, регулируется параметром *wal_level*.

- На уровне **minimal** журнал обеспечивает только восстановление после сбоя.
- На уровне **replica** в журнал добавляется информация, позволяющая использовать его для создания резервных копий и репликации. При репликации журнальные записи передаются на другой сервер и применяются там; таким образом создается и поддерживается точная копия (реплика) основного сервера.
- На уровне **logical** в журнал добавляется информация, позволяющая декодировать «физические» журнальные записи и сформировать из них «логические» записи о добавлении, изменении и удалении табличных строк. Это позволяет организовать логическую репликацию (рассматривается в соответствующих темах курсов DEV2 и DBA3).

Итоги

Буферный кеш существенно ускоряет работу,
уменьшая число дисковых операций

Надежность обеспечивается журналированием

Размер журнала ограничен благодаря контрольным точкам

Журнал удобен и используется во многих случаях

- для восстановления после сбоя
- при резервном копировании
- для репликации между серверами